

Design Pattern Report: Observer Pattern

Group 52

Nguyễn Đình Minh Huy (Student ID: 25125083)

July 20, 2026

Contents

1	Real-world Problem	2
2	Naive Solution (Without Design Pattern)	2
3	Problems with the Naive Solution	3
4	Introduction to the Observer Pattern	3
5	General Class Diagram	4
6	Class Diagram for the Bank Account Problem	4
7	Implementation of the Observer Pattern	5
8	Pros and Cons	7
8.1	Pros	7
8.2	Cons	7
9	Other Real-world Applications	7
9.1	Web Development	7
9.2	Mobile Development	8
10	Observer and Mediator Fusion	8
10.1	Case Study: Super Mario Game EventBus	9
11	Quiz	11
12	References	12

1 Real-world Problem

Consider a modern financial core banking system where transactions are performed on a `BankAccount`. When a deposit occurs, several peripheral systems need to be notified of the event immediately to execute their processes:

- **UI:** Updates the account balance display on the user's screen.
- **Email System:** Sends a transaction alert email to the customer.
- **Logger:** Writes transaction details to the security audit trail.
- **Fraud AI:** Scans transaction metadata to detect suspicious patterns.
- **Mobile App:** Delivers a push notification to the client's phone.
- **Analytics:** Records transaction metrics for business intelligence.

A customer or administrator must be able to register, enable, or disable these notification channels dynamically. The core challenge is establishing a system where a bank account can broadcast transaction updates to an arbitrary and dynamic set of observer systems without knowing their concrete classes or internal notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `BankAccount` class holds references to concrete notification system objects directly. Below is the C++ representation of this approach:

```
1 // Refer to Observer/src/naive.cpp for full implementation details
2 class NaiveBankAccount {
3 private:
4     double balance;
5     UI ui;
6     EmailSystem email;
7     Logger logger;
8     FraudDetector fraudDetector;
9     MobileApp mobileApp;
10    Analytics analytics;
11
12 public:
13     NaiveBankAccount(double initialBalance) : balance(initialBalance)
14     {}
15
16     void deposit(double amount) {
17         balance += amount;
18
19         // Direct, coupled method invocations on concrete systems
20         ui.update(balance);
21         email.sendEmail(amount);
22         logger.log(amount, balance);
23         fraudDetector.scan(amount);
24         mobileApp.pushNotification(amount);
25         analytics.trackActivity("Deposit", amount);
26     }
27 };
```

Listing 1: Naive Implementation of Bank Account Notifications

```
Terminal - bank_naive_demo

=====
BANK ACCOUNT DEMO: NAIVE VERSION
=====
Step 1: Initializing NaiveBankAccount with $1000.00...

Step 2: Depositing $250.00. Expecting notifications to ALL 6 systems...

[UI] Balance display updated to $1250.00
[Email] Alert sent: A deposit of $250.00 was made.
[Logger] Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI] Scan complete: Deposit of $250.00 is marked OK.
[Mobile App] Push: Account credited with $250.00
[Analytics] Metric recorded: Event [Deposit] for $250.00

Step 3: Attempting to detach Email and Mobile App...
Error: Cannot detach components. NaiveBankAccount is tightly coupled
to concrete notification systems.
=====
```

Figure 1: Console Output of the Naive Bank Account Demo (No Observer Pattern)

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveBankAccount` is tightly coupled to concrete classes like `UI`, `EmailSystem`, `Logger`, `FraudDetector`, `MobileApp`, and `Analytics`. It must know their names, structures, and exactly which update method to call on each.
2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new system (such as a compliance reporting system or a credit score monitoring engine), we must modify the core `NaiveBankAccount` class. This requires adding a new member field and updating the notification logic inside the `deposit` method.
3. **Violation of the Single Responsibility Principle (SRP):** The `BankAccount` class should focus solely on financial ledger operations (managing the balance and transactions). However, it is also burdened with orchestrating emails, logs, mobile push notifications, and security analysis.

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to `ConcreteObservers` and sends notifications.
- **ConcreteObserver:** Implements the `Observer` updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general Observer pattern structure:

- A Subject maintains a list of references to Observer objects.
- The Subject exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- Observer exposes the `update()` method interface.

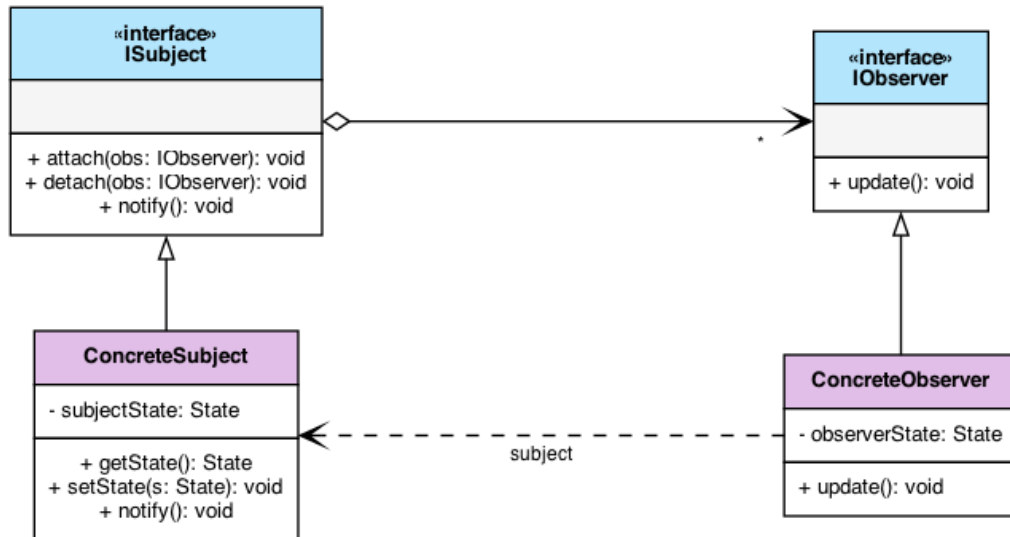


Figure 2: General Observer Design Pattern UML Diagram

6 Class Diagram for the Bank Account Problem

Applying the pattern to our problem:

- **BankAccount** acts as the **ConcreteSubject** (or subject coordinator).
- **AccountObserver** acts as the **Observer** interface.
- **UIObserver**, **EmailObserver**, **LoggerObserver**, **FraudAIObserver**, **MobileAppObserver**, and **AnalyticsObserver** act as the **ConcreteObservers**.

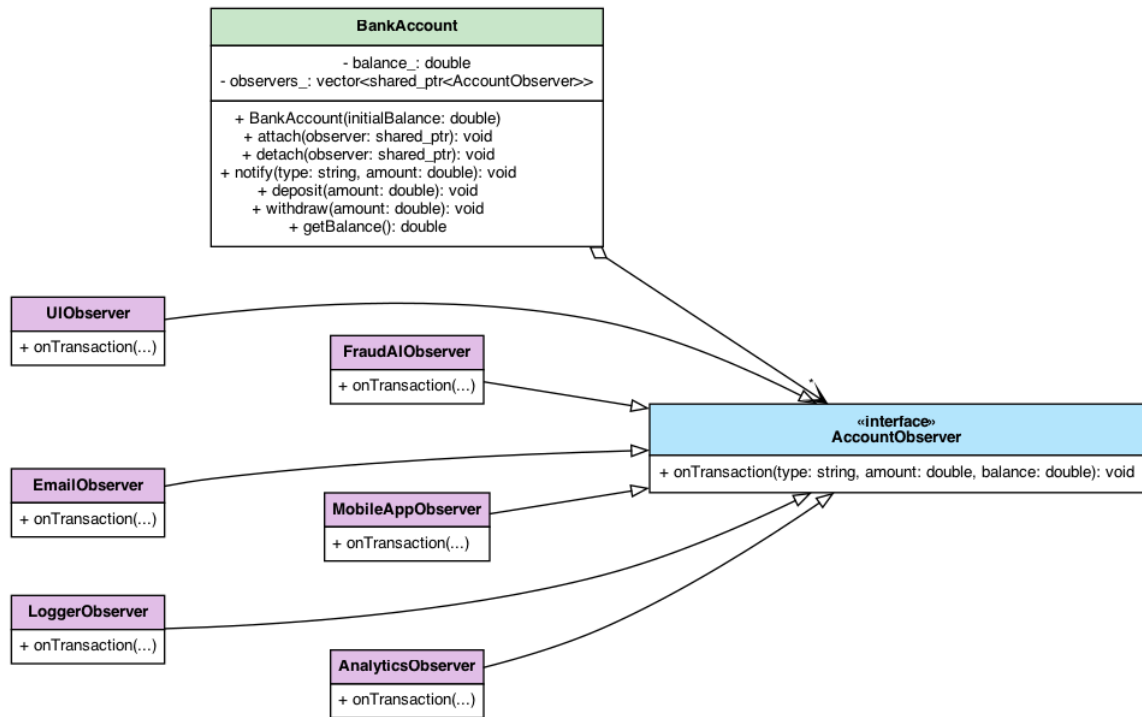


Figure 3: UML Class Diagram for Bank Account Notification System

7 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding or removing subscriber systems at runtime without modifying the core subject.

```

1 // Refer to Observer/src/pattern.cpp for full implementation details
2 class AccountObserver {
3 public:
4     virtual ~AccountObserver() = default;
5     virtual void onTransaction(const std::string& type, double amount,
6     double currentBalance) = 0;
7 };
8 class BankAccount {
9 private:
10     double balance;
11     std::vector<std::shared_ptr<AccountObserver>> observers;
12 public:
13     BankAccount(double initialBalance) : balance(initialBalance) {}
14     void attach(std::shared_ptr<AccountObserver> obs) { observers.
15     push_back(obs); }
16     void detach(std::shared_ptr<AccountObserver> obs) { /* Remove from
17     list */ }
18     void notifyObservers(const std::string& type, double amount) {
19         for (const auto& obs : observers) {
20             obs->onTransaction(type, amount, balance);
21         }
22     }
  
```

```

23
24     void deposit(double amount) {
25         balance += amount;
26         notifyObservers("Deposit", amount);
27     }
28 };
```

Listing 2: Refactored Observer Implementation

```

=====
BANK ACCOUNT DEMO: PATTERN VERSION
=====
==== RUNNING PATTERN DEMO (With Observer Pattern) ====
Step 1: Initializing BankAccount with $1000.00...

Step 2: Attaching observers...
- UIObserver (updates screen balance display)
- EmailObserver (sends transaction notification emails)
- LoggerObserver (writes to security/audit logs)
- FraudAIObserver (scans transaction amounts, warns if > $10,000)
- MobileAppObserver (sends push notification)
- AnalyticsObserver (records activity metrics)

Step 3: Depositing $250.00. Expecting notifications to ALL 6 systems...
[BankAccount] Deposited $250.00. New balance: $1250.00
[UI Observer] Balance display updated to $1250.00
[Email Observer] Alert sent: A deposit of $250.00 was made.
[Logger Observer] Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI] Scan complete: Deposit of $250.00 is marked OK.
[Mobile App] Push: Account credited with $250.00
[Analytics] Metric recorded: Event [Deposit] for $250.00

Step 4: Dynamically detaching Email and Mobile App observers...
(Customer disabled Email alerts and Mobile pushes in settings)
=====
```

Figure 4: Refactored Demo Output – Part 1

```

=====
BANK ACCOUNT DEMO: PATTERN VERSION
=====
Step 5: Depositing $12000.00. Expecting notifications to remaining systems...
(No Email, No Mobile push. Fraud AI alert should trigger for large amount)
[BankAccount] Deposited $12000.00. New balance: $13250.00
[UI Observer] Balance display updated to $13250.00
[Logger Observer] Transaction logged: Deposit of $12000.00, new balance: $13250.00
[Fraud AI] WARNING: Suspicious large activity detected: Deposit of $12000.00
[Analytics] Metric recorded: Event [Deposit] for $12000.00

Step 6: Customer re-enables Mobile App notifications in settings...
(Attaching MobileAppObserver back to BankAccount)

Step 7: Withdrawing $500.00. Expecting notifications to UI, Logger, Fraud AI...
[BankAccount] Withdrew $500.00. New balance: $12750.00
[UI Observer] Balance display updated to $12750.00
[Logger Observer] Transaction logged: Withdrawal of $500.00, new balance: $12750.00
[Fraud AI] Scan complete: Withdrawal of $500.00 is marked OK.
[Mobile App] Push: Account debited with $500.00
[Analytics] Metric recorded: Event [Withdrawal] for $500.00

Step 8: Attempting to withdraw $15000.00...
(Expecting Insufficient Funds failure alerts to remaining observers)
[BankAccount] Rejected: Insufficient funds for withdrawal of $15000.00
[UI Observer] Balance display updated to $12750.00
[Logger Observer] Transaction logged: FailedWithdrawal of $15000.00, new balance: $12750.00
[Fraud AI] WARNING: Suspicious large activity detected: FailedWithdrawal of $15000.00
[Mobile App] Alert: Withdrawal of $15000.00 failed (Insufficient Funds).
[Analytics] Metric recorded: Event [FailedWithdrawal] for $15000.00

==== PATTERN DEMO COMPLETED ====
=====
```

Figure 5: Refactored Demo Output – Part 2

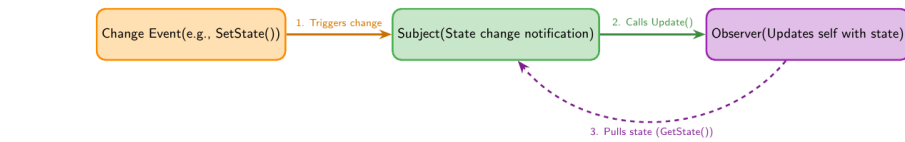


Figure 6: Sequential Process Flowchart of a Single Observer Notification

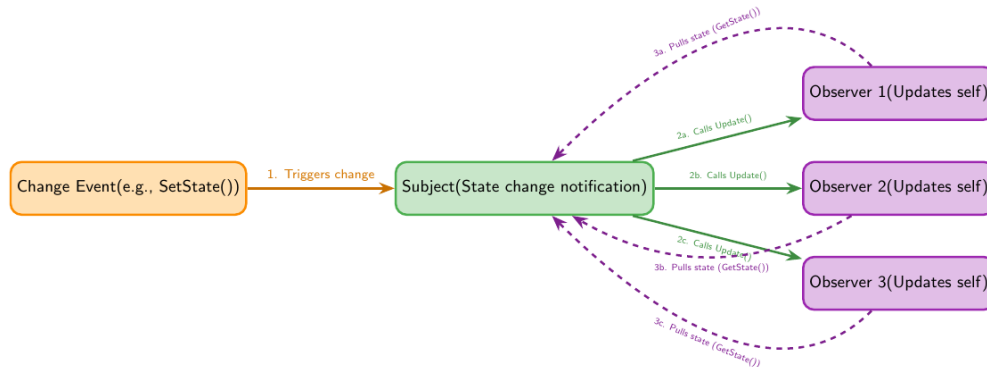


Figure 7: Sequential Process Flowchart with Multiple Observers (Pull Model)

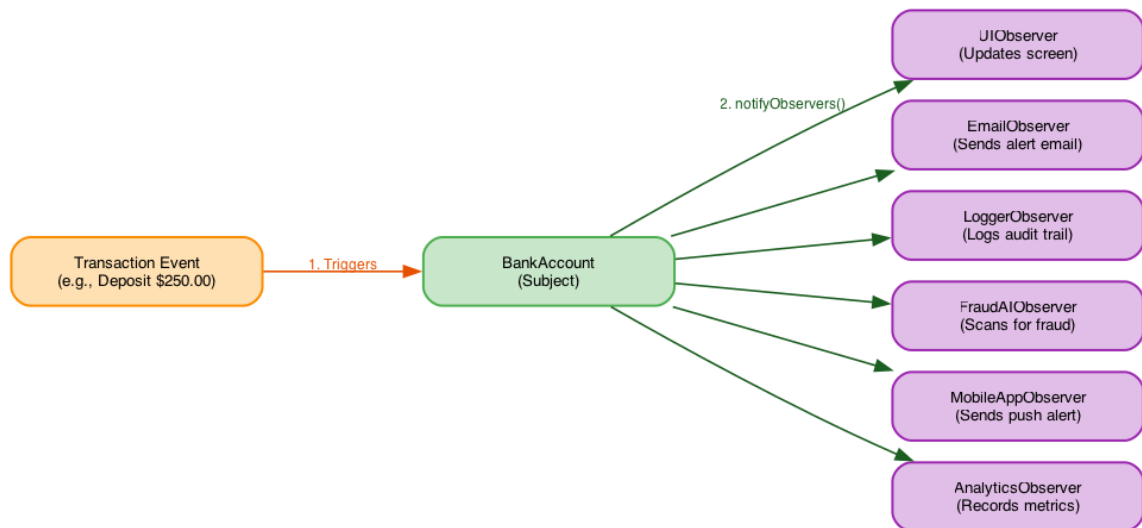


Figure 8: Process Flowchart for Bank Account Notification System (Push Model)

8 Pros and Cons

8.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered subscribers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

8.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.
- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.
- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

9 Other Real-world Applications

9.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.

- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

9.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

10 Observer and Mediator Fusion

In complex system architectures, the *Observer* and *Mediator* patterns are frequently fused together to handle coordination and communication between multiple components.

In a standard Mediator implementation, colleagues must explicitly call the mediator, and the mediator directly invokes actions on other colleagues. By fusing the two patterns:

- **Colleagues act as Subjects:** They maintain state and notify registered observers when changes occur, remaining completely unaware of who is listening.
- **The Mediator acts as an Observer:** It subscribes to the event streams of the Colleagues. When it receives an `update()` notification, it coordinates and executes the required reaction across other Colleagues.

This fusion achieves dynamic event-driven routing while preserving loose coupling between Colleagues.

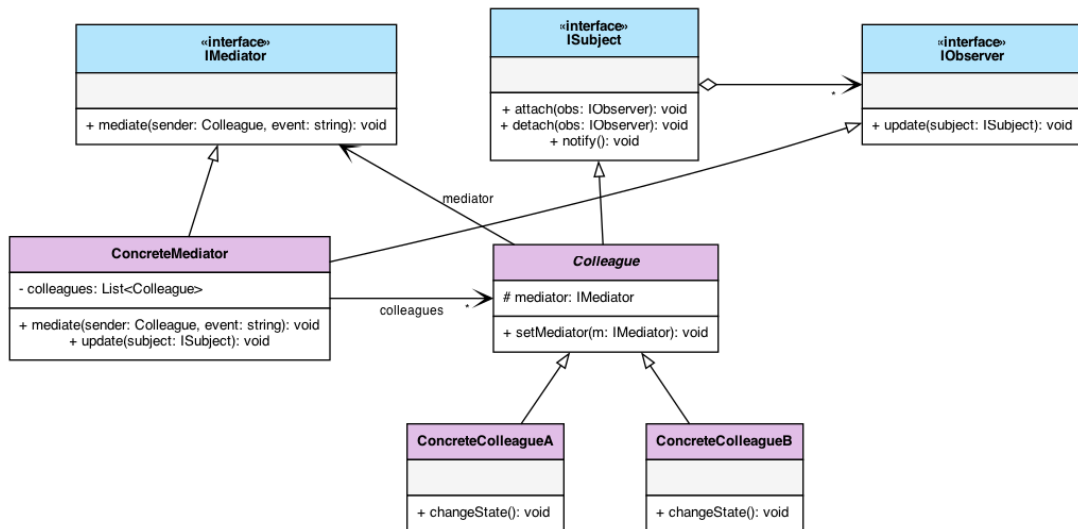


Figure 9: UML Class Diagram of the Observer-Mediator Fusion

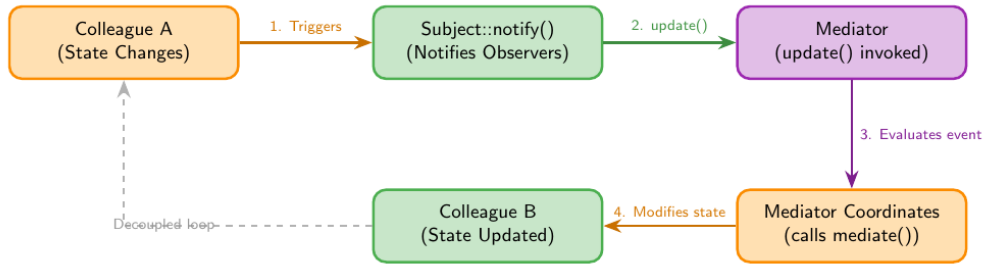


Figure 10: Process Flowchart of Event-Driven Mediation Flow

10.1 Case Study: Super Mario Game EventBus

In game development, handling interactions between numerous entities and systems (e.g., updating UI when a player dies, unlocking achievements when a coin is collected, playing sounds when blocks break) can easily lead to spaghetti code and tight coupling. To solve this, a decentralized event-routing system known as an **EventBus** is used.

The **EventBus** acts as a central singleton *Mediator*. Colleagues (such as the **Player** or **POWBlock**) publish events to the **EventBus** without knowing who reacts to them. Other systems (such as the **StatisticsTracker** or **AchievementManager**) register as *Observers* to specific event types.

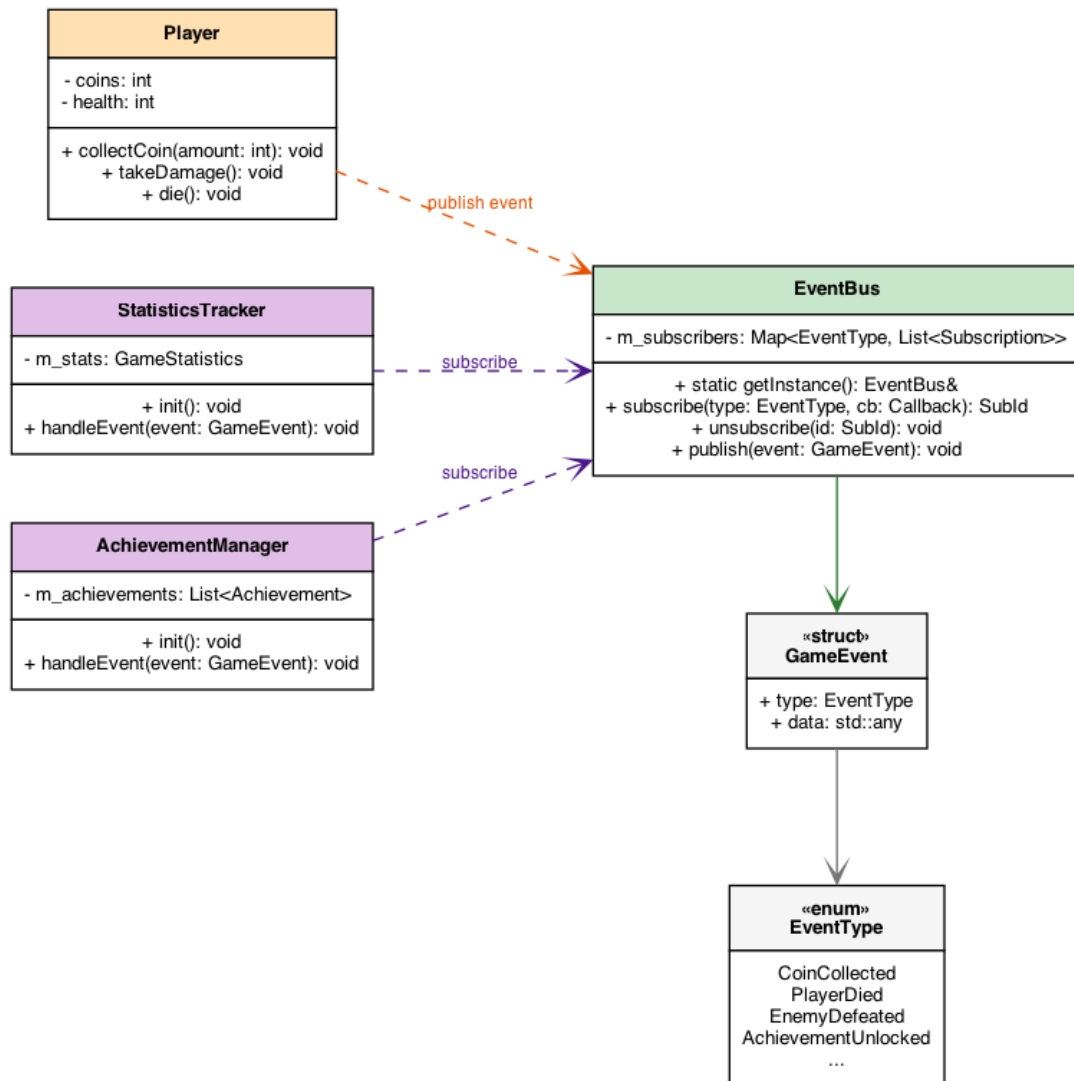


Figure 11: UML Class Diagram for Super Mario Game EventBus System

The C++ interface for the game's EventBus is implemented as follows:

```

1 // include/Core/EventBus.hpp
2 class EventBus {
3 public:
4     using Callback = std::function<void(const GameEvent &)>;
5     using SubscriptionId = size_t;
6
7     // Singleton Instance access
8     static EventBus &getInstance();
9
10    // Event operations (Mediating Hub)
11    SubscriptionId subscribe(EventType type, Callback callback);
12    void unsubscribe(SubscriptionId id);
13    void publish(const GameEvent &event);
14
15 private:
16     EventBus() = default;
17
18     struct Subscription {
19         SubscriptionId id;

```

```

20     Callback callback;
21 };
22
23     std::unordered_map<EventType, std::vector<Subscription>>
24     m_subscribers;
25     SubscriptionId m_nextId = 0;
26 };

```

```

=====
SUPER MARIO EVENTBUS SIMULATION DEMO
=====
[EventBus] System initialized. Singleton instance online.
[StatisticsTracker] Subscribed to CoinCollected
[StatisticsTracker] Subscribed to PlayerDied
[StatisticsTracker] Subscribed to ComboHit
[AchievementManager] Subscribed to CoinCollected
[AchievementManager] Subscribed to AchievementUnlocked

>> Player collects 1 gold coin...
[Player] Collected coin! Publishing event...
[EventBus] Publishing: CoinCollected (data: 1)
[StatisticsTracker] Received CoinCollected. Total coins: 1
[AchievementManager] Received CoinCollected. Evaluating achievements...

>> Player hits a POW Block!
[POWBlock] Broken! Triggering screen shake and screen enemy flip...
[EventBus] Publishing: BlockBroken (data: POWBlock*)
[EventBus] (No subscribers registered for EventType::BlockBroken)

>> Player defeats Goombas (x5 Combo Hit!).
[Player] Combo hit count: 5. Publishing event...
[EventBus] Publishing: ComboHit (data: 5)
[StatisticsTracker] Received ComboHit. Max combo: 5
[StatisticsTracker] Tracking combat metrics...

>> Player collects 99 gold coins (Milestone!).
[Player] Collected 99 coins! Publishing event...
[EventBus] Publishing: CoinCollected (data: 99)
[StatisticsTracker] Received CoinCollected. Total coins: 100
[AchievementManager] Received CoinCollected. Evaluating achievements...
[AchievementManager] *** UNLOCKED ACHIEVEMENT: Golden Hoarder! ***
[EventBus] Publishing: AchievementUnlocked (data: Golden Hoarder)

>> Player falls into pit.
[Player] Died! Publishing event...
[EventBus] Publishing: PlayerDied (data: Player*)
[StatisticsTracker] Received PlayerDied. Death count: 1
=====

```

Figure 12: Console Output Simulation of EventBus Event-Driven Mediation

11 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?
 - A) Creational
 - B) Structural
 - C) Behavioral

- D) Architectural
2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new subscribers?
- A) Liskov Substitution Principle
 - B) Single Responsibility Principle
 - C) Interface Segregation Principle
 - D) Open-Closed Principle
3. What is the “Lapsed Listener” problem in the Observer pattern?
- A) The subject stops listening to updates.
 - B) Observers are not notified because the subject crashed.
 - C) An observer is deleted without unsubscribing, causing memory leaks/crashes.
 - D) Multiple subjects occupy the same listener list.
4. Which of the following is a key advantage of the Observer pattern?
- A) It ensures observers are updated sequentially.
 - B) It decouples the subject from concrete observer details.
 - C) It reduces memory overhead of notifications.
 - D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

12 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer*. <https://refactoring.guru/design-patterns/observer>